# 6.2 Separate routing from display

Keep the entry script, stylesheet, and all templates from 6.1. We will change only where the PHP responsibilities live. The browser should show exactly the same two pages afterward.

## Create DefaultController

Create `src/Tutorials/Class6/DefaultController.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

class DefaultController
{
    private const PATH_TO_TEMPLATES = __DIR__ .
'/../../../templates/tutorials/class6';

    private Environment $twig;

    public function __construct()
    {
        $loader = new FilesystemLoader(self::PATH_TO_TEMPLATES);
        $this->twig = new Environment($loader, [
            'autoescape' => 'html',
            'strict_variables' => true,
        ]);
    }

    public function home(): void
    {
        $template = 'home.html.twig';
        $args = [];
        echo $this->twig->render($template, $args);
    }

    public function contact(): void
    {
        $template = 'contact.html.twig';
        $args = [];
        echo $this->twig->render($template, $args);
    }
}
```

DefaultController owns the Twig setup and the page-generation methods. It does not read the action query parameter and has no run method. Its name describes the basic pages it handles, not a special PHP feature.

home and contact are now **public** because Application is a different class and must call them. In 6.1, private was appropriate because those methods were called inside Application itself. Making a method public does not automatically create a URL route; our switch still determines which method runs.

## Replace Application

Replace `src/Tutorials/Class6/Application.php` with:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

class Application
{
    public function run(): void
    {
        $defaultController = new DefaultController();

        $action = $_GET['action'] ?? 'home';
        if (!is_string($action)) {
            $action = 'home';
        }

        switch ($action) {
            case 'contact':
                $defaultController->contact();
                break;
            case 'home':
            default:
                $defaultController->home();
        }
    }
}
```

Application now acts as the front controller: inspect the request and delegate to a known page method. Remove its former Twig imports, property, template-path constant, constructor, and private page methods by using this complete replacement.

DefaultController chooses the template, supplies its context, and echoes the rendered result. Twig describes the HTML. Application uses a DefaultController object; it does not inherit from it.

## Verify the refactoring

Repeat Home, Contact, unknown action, and array-shaped action requests from 6.1. Titles, content, navigation, and fallbacks should be unchanged. This is a refactoring: the code structure changes while the visible behavior stays the same.